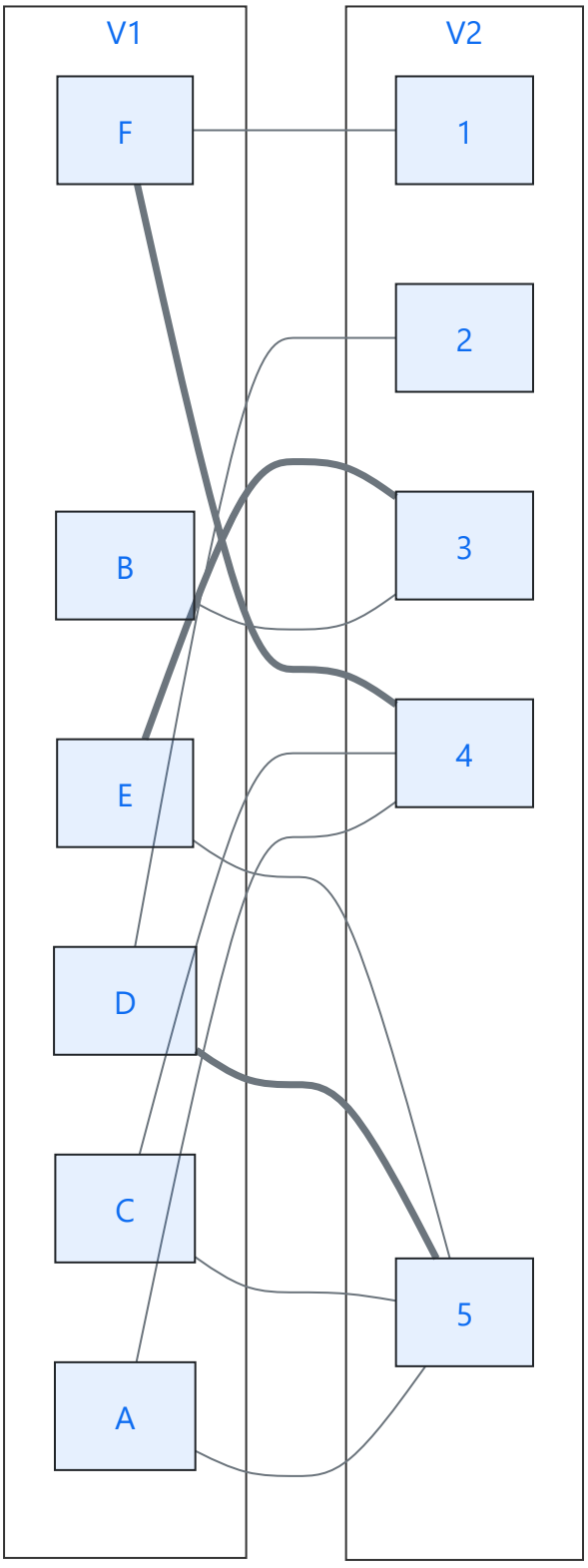
Refresher on matches and the algorithm using this video: https://youtu.be/lM5eIpF0xjA

Also I am using mermaid for the graphs, I dont know why the graph is out of order compared to the original chart but the graph below is isomorphic to the one in the homework pdf (easiest to see when you do vertical A-F on the left and 1-5 on the right from the homework)



This means our original match edges are: $\{D--5, E--3, F--4\}$

'

We can prove that this state is possible by going through a couple iterations of the algorithm. The algorithm in the video I attatched is:

```
M = \phi

repeat:

  use BFS to build aternating level graph, rooted at unmatched vertices in set V1

  Augment current matching M with maximal set of vertex disjoin shortest length paths (DFS)

Until there are no more augmenting paths

return M
```

In the video as well, the halted the BFS when reaching a level where there are vertices not in the match already. So in our case, the algorithm if started on node D would halt the BFS after one layer with possibilites of $5, 2$. So we could add the edge $D - -5$ to M

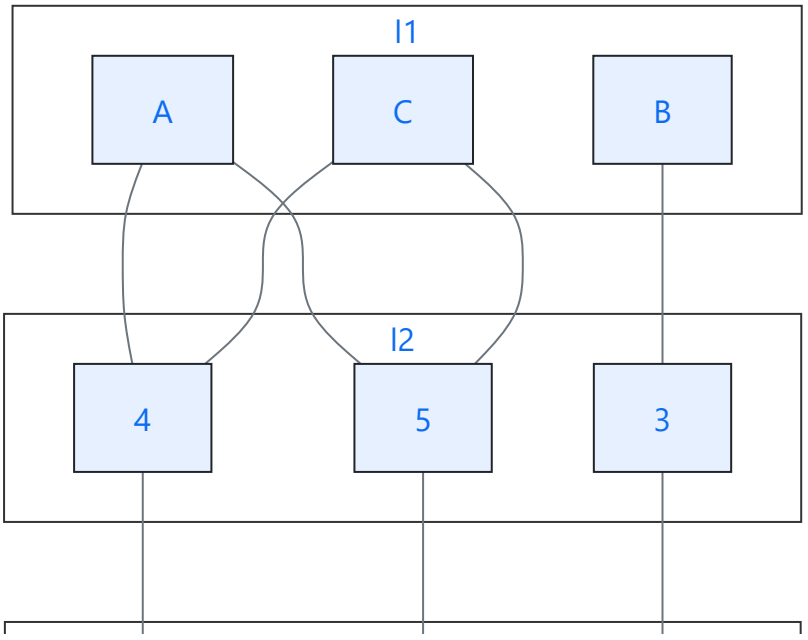
The next iteration could look at E which again halts the BFS after one layer and could augment the path $E - -3$ to M. And the third iteration (and the last one to get to our current set)$ would look at F and augment the path $F - -4$.

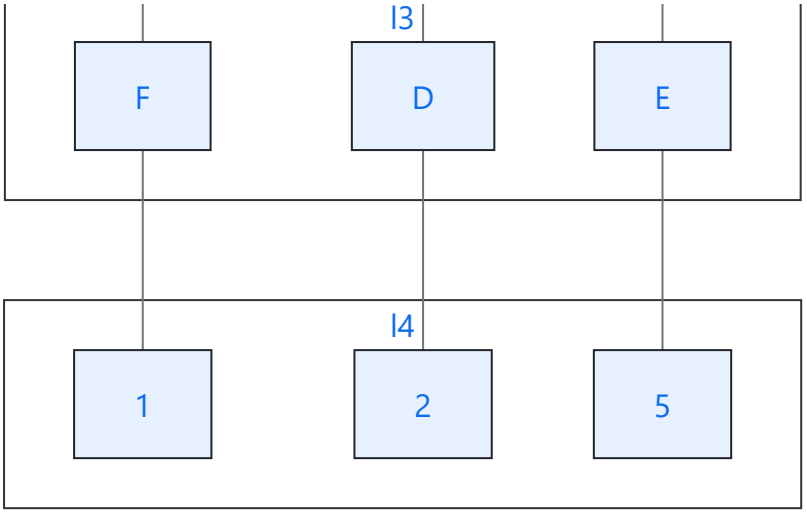
This possibility would end at $M = \{F - -4, D - -5, E - -3\}$ which are the exact bolded edges outlined by our original problem. Therefore it is possible for the algorithm to produce this match at the end of a few iterations

2

First we perform BFS on the vertices not in our match but in V_1, which are the nodes B, A, C

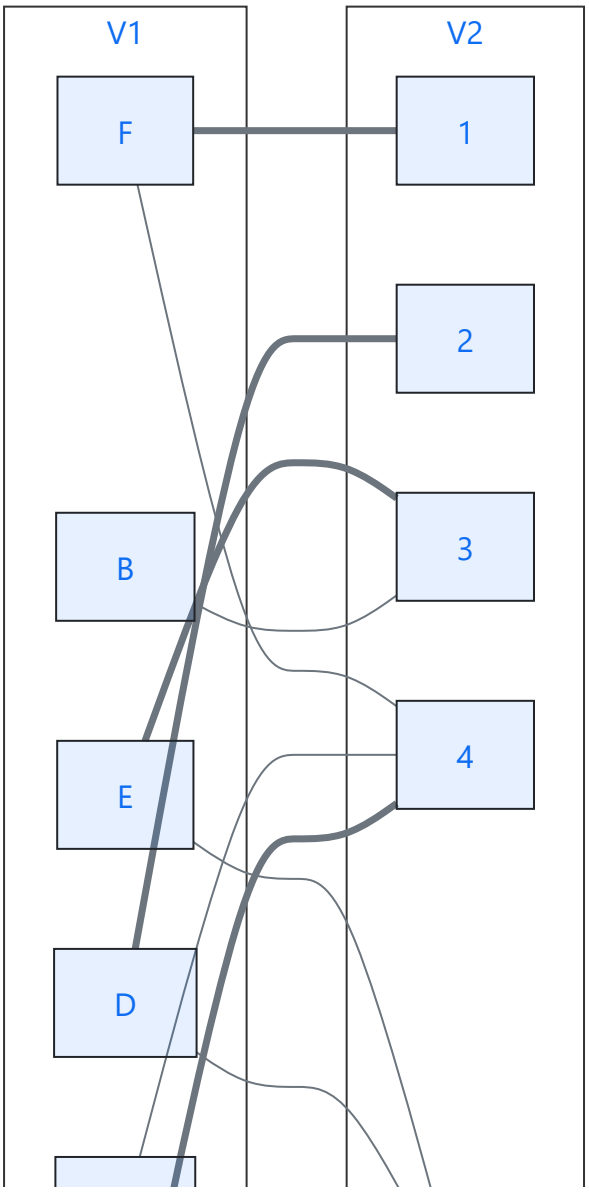
Ill make this step clear with a graph below:

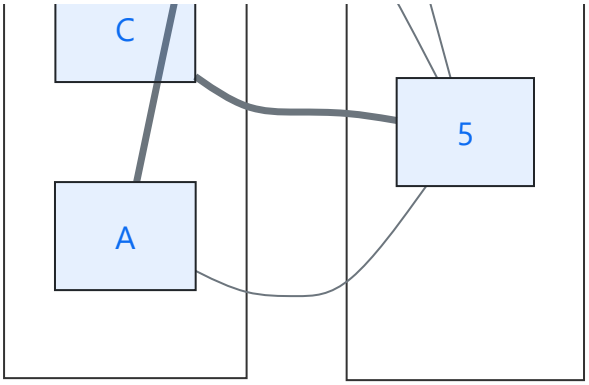




Now that we are here we can perform DFS on our nodes in layer 1 to create a few augmented paths: $A-4-F-1, C-5-D-2$. Now we aren't using the path with B because there is no vertex disjoint path.

Thus our new match edges are: $\{A--4, F--1, C--5, D--2, E--3\}$ which looks like this:





And we actually know that this is a maximal match because in a bipartite graph the maximal match M is at most $\min(|V_1|, |V_2|)$ which in our case it is because we have a match of size 5

Therefore our algorithm stops here because there are no more augmenting paths.

3

The trivial case is for M to have a disjoint edge because then we simply remove that edge from M and add it to N to create our M', N'

The idea here for the nontrivial case is to use the same sort of augmenting path trick to create a M' and N' that follow those rules.

For some edge $(u, v) \in N$ with constraints that $(u, v) \backslash \text{nin} M \wedge \exists e_1, e_2 \in M(u \in e_1, v \in e_2)$

Thus we can now create $M' = M - e_1 - e_2 + (u, v), N' = N + e_1 + e_2 - (u, v)$ and we know our set union and intersection will hold still because those operators are indescriminate of which set indiviual elements are. The bigger issue comes from proving that there exists the (u, v) and e_1, e_2

The idea being that if there does not exist some (u, v) and e_1, e_2 then there would be an edge in M that is disjoint from N and fall into the trivial case. Thus every edge in M must have at least one vertex in N and since $M > N$ we know that at least one edge in N must contain 2 vertices of two different edges in M

Therefore we know such set of edges exist and that we can create M' abd N' that satisfy the constraints.